VS Code (GitHub Copilot Chat) — Configuration Report

Prepared for **Murpheux** · 28 September 2026 · Verified against **VS Code 1.139.1** (installed) and Copilot Chat 0.68
Folder: `~/agent-configs/vscode/` → `<profile>/` (settings, mcp.json, prompts) and `~/.copilot/` (shared core)

1. Summary

VS Code 1.139 runs two agent harnesses:

- **Local** reads profile settings, prompt files and `mcp.json`. It is slated for removal.
- **Copilot** reads only `~/.copilot` and `~/.claude`.

The configuration therefore splits in two. The shared `~/.copilot` core (instructions, skills, guard hook, MCP) works in both harnesses and the CLI. VS Code-only pieces go in the profile: approvals, sandbox, prompt files and `mcp.json`.

The router design had to adapt to one VS Code rule: **a subagent priced above its caller is refused**. The Router sits on the Medium tier and runs cheaper or equal-tier subagents. The premium Architect and Security run as **handoff buttons**, which you confirm.

8	16	20	17	5
VS Code agents	prompt files	skills	terminal rules	MCP servers

2. Sources used

- `microsoft/vscode-docs`: agent customization (instructions, prompt files, custom agents, skills, hooks, MCP, approvals, sandboxing, subagents, harnesses, language models) and release notes v1_118–v1_140.
- `microsoft/vscode` source: the prompt validator, subagent tool, settings registry and Agent Host converters.
- **github/awesome-copilot** and **awesome-copilot.github.com**; `github/docs` hooks reference and custom-agent configuration.

3. What was built

File	Purpose
`~/.copilot/agents/vscode-*.agent.md`	Router, Explorer, Reviewer, Planner, Developer, SRE, Architect, Security. All are `target: vscode`, with model fallback arrays, `reasoning-effort`, tool aliases, an `agents` allow-list and `handoffs`
`~/.copilot/instructions/murpheux.instructions.md`	the same global instructions file as the CLI folder (`applyTo: "**"`)
`~/.copilot/skills/`, `hooks/`, `mcp-config.json`, `mcp.op.env`	shared core: 20 skills, the guard hook (Copilot hook format, which the Local harness also reads), MCP for the Copilot harness, and 1Password references
`<profile>/settings.json` (merged)	AGENTS.md on, `useClaudeMdFile` off, maxRequests 60, nested subagents, terminal and edit auto-approve rules, sandbox, hooks, plugins, commit/review instructions
`<profile>/mcp.json`	github (OAuth), context7 (via `op run`, so no `${input:}` is needed and the server is still forwarded to the Copilot harness), mslearn, awsdocs, terraform

`<profile>/prompts/*.prompt.md`	16 prompt files (Local harness), each bound to the right agent. Cheap ones pin GPT-6 Luna

4. Routing & model tiers

Agent	Model fallbacks	How it's reached
Router	Claude Sonnet 5 → GPT-6 Sol → Auto	agent picker. Tools: agent, read, search, web, todo
Explorer · Reviewer	GPT-6 Luna → Claude Haiku 4.5 → Auto	subagent (cheapest)
Planner	GPT-6 Sol → Claude Sonnet 5 · medium	subagent
Developer · SRE	Claude Sonnet 5 → GPT-6 Sol · medium	subagent, or the handoff "Implement with Developer"
Architect · Security	Claude Opus 5.5 → GPT-6 Sol · high	**handoff only** (premium tier)

Billing is in AI credits (token-based since 2026-06-01; 1 credit = $0.01). Auto gets a 10% discount. Confirm the exact model names in *Chat: Manage Language Models*; each agent falls back to Auto if a name differs.

5. Guardrails

1. **Terminal auto-approve**: only read-only and validation commands run automatically. Push, IaC apply/destroy, cluster mutations, cloud CLIs and `.ps1` always ask. Workspace npm scripts are no longer auto-approved.
2. **Guard hook**: this is what actually *blocks*. Terminal rules only ask, and VS Code's docs say they are not a security boundary.
3. **Edit protection**: `.env*`, state and key files, workflows and `.ssh` always ask. VS Code already protects `.git/**`, lockfiles and `.vscode/*.json`.
4. **Sandbox**: on for macOS/Linux/WSL2 (WSL2 needs bubblewrap and socat); off on Windows, where it is experimental.
5. **Prompt injection**: web fetch is not eligible for auto-approval; `chat.useClaudeMdFile` is off to avoid duplicated or unexpected instructions.

6. From awesome-copilot

- **Install** via `@agentPlugins` (the marketplace is built in): csharp-dotnet-development, aws-/azure-cloud-development, devops-oncall, structured-autonomy.
- **Adopted natively**:
  - rug-orchestrator → Router;
  - se-security-reviewer (made read-only), se-system-architecture-reviewer, aws-/azure-principal-architect → Security/Architect;
  - terraform-iac-reviewer and platform-sre-kubernetes → iac-review and SRE;
  - tool-guardian and secrets-scanner hooks → guard;
  - draw-io, threat-model-analyst, ADR and incident-postmortem skills → shared skills.
- **Avoided**: always-on `security-and-owasp.instructions.md` (1,088 lines), kept as an on-demand skill instead.

7. Gotchas handled

- Prompt files don't run in the Copilot harness, so every command also exists as a skill (in the CLI folder).
- `~/.copilot` doesn't settings-sync, and on Windows + WSL each harness reads its own host. Run `install.ps1` on Windows *and* `install.sh` in WSL.
- Profile MCP servers run on Windows even in WSL windows. The Mermaid renderer tool is Local-only.

- Agents from `~/.claude/agents` also appear in the Copilot harness if Claude Code is installed; see the final report's coexistence notes.

8. Validation

The settings fragment (JSONC), `mcp.json`, all agents and all prompt frontmatter parse. The installer smoke test merged settings while keeping an existing key (`editor.fontSize`), installed 16 prompt files and `mcp.json`, and confirmed all 8 agents are `target: vscode`. `install.ps1` parses cleanly.

9. Install

```
cd ~/agent-configs/vscode && ./install.sh              # macOS / Linux / WSL (~/.copilot + remote machine
settings)
pwsh -File install.ps1                                  # Windows (copy the folder to a local drive first)
# Reload VS Code → Chat → pick "Router" → Chat: Diagnostics to confirm agents/tools/hooks
```